

Arbeitsblatt 7: Vom Schwimmer zur Anzeige

Analog messen, digital anzeigen – der Füllstandsgeber | 4. September 2026



Lösungen & Erklärungen zu allen Aufgaben:

hbraak.github.io/fachpraxis/arduino/lsg7.html

Erst selbst versuchen – die Seite hilft beim Nachvollziehen, nicht beim Abschreiben.

Material – bitte zuerst abhaken

Bauteil	vorh.	Bauteil	vorh.
Arduino Uno + USB-Kabel	☐	5 LEDs + je 330 Ω	☐
Steckbrett	☐	Steckbrücken / Kabel	☐
Potentiometer, linear	☐	Servo SG90 (<i>nur Zusatz</i>)	☐

Teil 1 läuft **ohne jedes Bauteil**. Fehlt etwas für die späteren Teile, meldet euch – Teams werden gemischt.

Die Leitfrage

Ein Tank, ein Regenrückhaltebecken, ein Klärbecken: Überall muss eine Steuerung wissen, *wie voll* der Behälter ist. Der Füllstand ändert sich **stufenlos**. Ein Mikrocontroller kennt aber nur **Zahlen**. Wo genau passiert der Übergang – und was geht dabei verloren?

Teil 1 – Analog und digital: Wiederholung ohne Bauteile

Steckt **nur ein Kabel** in den Arduino: ein Ende an A0. Ladet diesen Sketch und öffnet den **Seriellen Monitor** (9600 Baud).

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  Serial.println(analogRead(A0)); // Rohwert des ADC
  delay(200);
}
```

1. Steckt das freie Kabelende nacheinander an 5V, an GND, und lasst es dann **lose in der Luft** hängen. Notiert:

Kabelende an	Rohwert	Spannung an A0
5V	_____	_____
GND	_____	_____
lose (offen)	_____	—

2. Der ADC hat **10 Bit**, also $2^{10} = 1024$ Stufen im Bereich 0 V bis 5 V. Eine Stufe ist damit

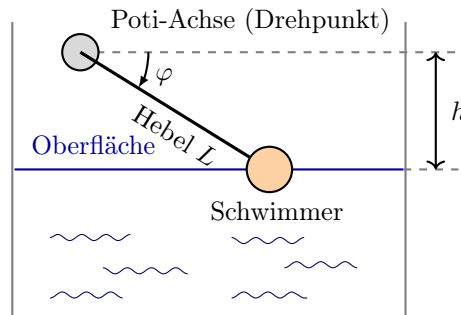
$$\Delta U = \frac{5 \text{ V}}{1024} = \text{_____ mV.}$$

3. Warum liefert das **lose** Kabel keinen festen Wert? _____

4. Erklärt in einem Satz den Unterschied *analog* ↔ *digital* am Beispiel dieser Messung:

Teil 2 – Der Schwimmergeber: die Physik dahinter

So misst jeder Autotank und jedes Regenrückhaltebecken seinen Füllstand: Ein **Schwimmer** sitzt an einem **Hebel**, dessen Drehpunkt auf der Achse eines **Potentiometers** liegt.



Warum bleibt der Schwimmer an der Oberfläche? Er taucht so weit ein, bis die Auftriebskraft die Gewichtskraft ausgleicht:

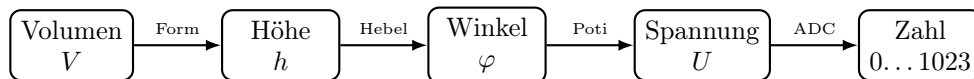
$$F_A = \rho_{\text{Wasser}} \cdot g \cdot V_{\text{verdrängt}} = F_G = m \cdot g$$

Die verdrängte Menge und damit die **Eintauchtiefe bleibt konstant** – egal wie hoch das Wasser steht. Der Schwimmer folgt also der Oberfläche.

5. Ein Schwimmer hat $V = 40 \text{ cm}^3$ und $m = 12 \text{ g}$. Welchen Anteil seines Volumens taucht er ein? ($\rho_{\text{Wasser}} = 1 \text{ g/cm}^3$) _____

Die Übertragungskette

Vom Wasserstand bis zur Zahl im Speicher durchläuft die Information **fünf** Stationen:



Der Hebel überträgt die Höhe **nicht** proportional, sondern über den Sinus:

$$h = L \cdot \sin \varphi$$

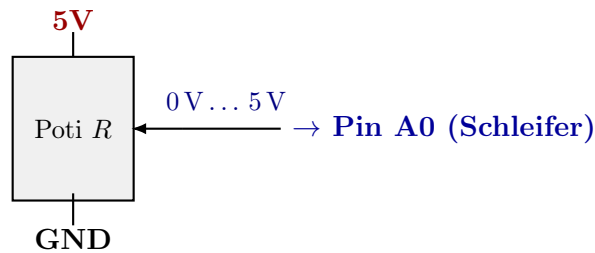
6. Der Geber eines Regenrückhaltebeckens hat einen Hebel der Länge $L = 35 \text{ cm}$. Berechnet die Höhe h über dem Drehpunkt:

φ	-50°	-25°	0°	25°	50°
h in cm	_____	_____	_____	_____	_____

7. Wie groß ist der Höhenzuwachs von 0° auf 25° (_____) – und wie groß von 25° auf 50° (_____)?
8. Welche der vier Übertragungen in der Kette oben ist damit **nicht** linear? _____

Teil 3 – Der Geber am Arduino

Der echte Geber sitzt im Becken. Auf dem Tisch **simulieren** wir ihn mit dem Poti: Es ist elektrisch exakt dasselbe Bauteil – ein **verstellbarer Spannungsteiler**. Außen 5 V und GND, in der Mitte der Schleifer an A0.



9. Baut die Schaltung auf, ladet den Sketch aus Teil 1 erneut und öffnet den **Seriellen Plotter**. Dreht langsam von Anschlag zu Anschlag und notiert:

	linker Anschlag	Mittelstellung	rechter Anschlag
Rohwert	_____	_____	_____

In Mittelstellung muss ungefähr **512** stehen – nur dann ist euer Poti *linear* und als Winkelgeber brauchbar.

Teil 4 – Die Kennlinie des Prototyps

Vorne steht ein **Schwimmergeber-Prototyp**. Wir vermessen ihn gemeinsam, so wie ein Techniker eine Anlage bei der Inbetriebnahme aufnimmt: Pegel einstellen, Rohwert ablesen, Wertepaar notieren.

Messpunkt	1	2	3	4	5	6	7
Pegel h in cm	_____	_____	_____	_____	_____	_____	_____
Rohwert (ADC)	_____	_____	_____	_____	_____	_____	_____

10. Tragt die Wertepaare ein und verbindet sie. **Schreibt zuerst eure Skala auf die kurzen Striche** an den Achsen – wählt sie so, dass eure Messwerte das Feld möglichst ganz ausfüllen.



11. Berechnet die **Differenz** von Messpunkt zu Messpunkt und tragt sie ein:

Schritt	1→2	2→3	3→4	4→5	5→6	6→7
Δ Rohwert	_____	_____	_____	_____	_____	_____

Werden die Differenzen **größer, kleiner oder bleiben sie gleich?** _____

12. Ist die Kennlinie eine **Gerade**? Vergleicht mit Aufgabe 7 und begründet: _____

Teil 5 – Kalibrieren: von 0–1023 auf 0–100 %

13. Lest aus *eurer* Messreihe ab, welchen Rohwert der Prototyp liefert – **nicht raten!**
 Becken leer: _____ Becken voll: _____
14. Tragt die Werte des Gebers in den Sketch ein. `constrain()` verhindert Werte außerhalb von 0–100 %. Dreht dann am Poti: Es spielt jetzt den Schwimmer.

```
int LEER = ____;    // <-- Rohwert bei "leer"
int VOLL = ____;    // <-- Rohwert bei "voll"

void setup() {
  Serial.begin(9600);
}

void loop() {
  int roh      = analogRead(A0);
  int prozent  = map(roh, LEER, VOLL, 0, ____);    // <-- ergaenzen
  prozent      = constrain(prozent, 0, 100);
  Serial.println(prozent);
  delay(100);
}
```

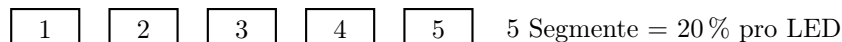
15. Dreht das Poti langsam über den ganzen Bereich. Ab welcher Stellung bleibt die Anzeige bei 0 % bzw. bei 100 % stehen – und was tut `constrain()` dort? _____
16. Warum darf man **nicht** einfach `map(roh, 0, 1023, 0, 100)` schreiben? Rechnet mit *eurem* Leer-Rohwert nach, welchen Prozentwert das **leere** Becken dann anzeigen würde:

$$\text{Anzeige} = \frac{\text{LEER} \cdot 100}{1023} = \text{_____} \%$$

Was wäre daran fatal? _____

Teil 6 – Anzeigen: der Balken

Jetzt kommt das andere Ende der Kette. Steckt **5 LEDs** an die Pins 2 bis 6 (jede über 330 Ω nach GND) – das ist eure Füllstandsanzeige.



```
void loop() {
  int roh      = analogRead(A0);
  int prozent  = constrain(map(roh, LEER, VOLL, 0, 100), 0, 100);

  int segmente = prozent / 20;    // 0 ... 5
  for (int i = 0; i < 5; i++) {
    digitalWrite(2 + i, i < segmente ? HIGH : LOW);
  }
  Serial.println(prozent);
  delay(100);
}
```

17. Ergänzt `setup()` so, dass die Pins 2–6 Ausgänge sind (Schleife verwenden!).

18. Wie viele Prozent Füllstand entsprechen **einer** LED? _____
Wie viele ADC-Stufen nutzt euer Prototyp überhaupt? VOLL – LEER = _____
Wie viele Prozent entsprechen damit **einer ADC-Stufe**? _____
19. **Die entscheidende Frage:** Der ADC liefert 1024 Stufen, die Anzeige zeigt 5. Wozu misst man dann überhaupt so fein? _____

20. Formuliert als Merksatz, wie sich Messauflösung und Anzeigeauflösung zueinander verhalten müssen: _____

Zusatzaufgaben

21. **Alarm:** Erweitert das Programm so, dass die oberste LED **blinkt**, sobald der Füllstand über 90 % steigt (Überlaufwarnung im Rückhaltebecken).
22. **Zeigerinstrument (nur mit Servo):** Ersetzt den LED-Balken durch einen Servo als analoge Anzeigenadel: `winkel = map(prozent, 0, 100, 0, 180)`. Vergleicht: Welche Anzeige ist feiner – und passt das zur Messgenauigkeit?
23. **Nichtlinearität korrigieren:** Aus $h = L \sin \varphi$ folgt $\varphi = \arcsin(h/L)$. Recherchiert, wie man in der Praxis statt einer Formel eine **Kalibriertabelle** (Stützstellen) benutzt, und skizziert, wie das im Code aussähe.
24. **Transfer:** Warum steht die Tankanzeige im Auto gefühlt ewig auf „voll“ und fällt im letzten Viertel schnell? Nennt **zwei** Ursachen aus der Übertragungskette.
25. **Spielt der Ohm-Wert eine Rolle?** In der Sammlung liegen Potis mit 470Ω und mit $10\text{ k}\Omega$. Begründet mit der Spannungsteilerregel, warum beide *denselben* Rohwert liefern. Berechnet anschließend für beide den Querstrom $I = U/R$ bei 5 V und überlegt, welcher Wert im Dauerbetrieb sparsamer ist.

Notizen zu den Zusatzaufgaben:



Lösung & Erklärungen:
hbraak.github.io/fachpraxis/arduino/lsg7.html